CÂU HỎI ÔN TẬP

Từ mã nguồn C đến chương trình đang chạy trên vi điều khiển

Đề bài

Cho một hệ thống nhúng sử dụng vi điều khiển STM32F103C8 (lõi ARM Cortex-M3, kiến trúc ARMv7-M) với các thông số sau:

- Bộ nhớ Flash dung lượng 64 KB, bắt đầu tại địa chỉ `0x0800 0000`.
- Bộ nhớ SRAM dung lượng 20 KB, bắt đầu tại địa chỉ `0x2000 0000`.
- Chân cấu hình khởi động được đặt để nạp chương trình từ Flash chính (Flash được ánh xạ phụ — alias — sang địa chỉ `0x0000 0000` tại thời điểm khởi động).
- Bộ công cụ biên dịch: `arm-none-eabi-gcc`, kèm tệp khởi động (startup file) và tệp kịch bản liên kết (linker script, phần mở rộng `.ld`) do STM32CubeMX sinh ra.

Chương trình nguồn gồm hai tệp `main.c` và `cambien.c`, trong đó `main.c` có nội dung rút gọn như sau:

```
#include <stdint.h>

const char THONG_BAO[] = "He thong san sang";  // hang so
uint32_t soLanDo = 0;          // toan cuc, khoi tao = 0
uint32_t nguongCanhBao = 750;  // toan cuc, khoi tao khac 0

uint32_t binhPhuong(uint32_t x) {
    return x * x;
}

int main(void) {
    uint32_t kq = binhPhuong(5);
    while (1) { }
}
```

Câu 1. Quá trình từ chương trình C sang chương trình trên chip (2 điểm)

a. Trình bày bốn giai đoạn mà bộ công cụ `arm-none-eabi-gcc` thực hiện để chuyển `main.c` và `cambien.c` thành một tệp thực thi `.elf`. Nêu rõ đầu vào và đầu ra (dạng tệp) của từng giai đoạn.

b. Tệp kịch bản liên kết (`.ld`) giữ vai trò gì mà nếu thiếu nó thì chương trình không thể chạy được trên chip, trong khi biên dịch cho máy tính cá nhân lại gần như không bao giờ cần quan tâm tới tệp này?

c. Sau khi liên kết, ba biến `THONG_BAO`, `soLanDo` và `nguongCanhBao` trong đoạn mã trên được xếp vào ba section khác nhau. Hãy chỉ ra tên từng section, giải thích vì sao trình liên kết lại phân loại như vậy, và cho biết mỗi section chiếm chỗ trong Flash, trong SRAM, hay trong cả hai.

d. Giải thích cặp khái niệm địa chỉ nạp (LMA — Load Memory Address) và địa chỉ chạy (VMA — Virtual Memory Address). Section nào trong ý c có LMA khác VMA, và điều đó dẫn tới hệ quả gì cho đoạn mã khởi động?

Câu 2. Khởi động hệ thống và bảng vector ngắt (3 điểm)

a. Ngay sau khi tín hiệu reset được giải phóng, lõi Cortex-M3 tự động đọc hai từ nhớ 32-bit đầu tiên của bảng vector ngắt. Cho biết mỗi từ nhớ đó chứa giá trị gì, và giá trị đó được nạp vào thanh ghi nào của CPU.

b. Với cấu hình bộ nhớ nêu ở phần bối cảnh, hãy tính giá trị cụ thể mà từ nhớ thứ nhất phải chứa, và giải thích vì sao lại là giá trị đó.

c. Giả sử từ nhớ thứ hai của bảng vector chứa giá trị `0x0800 0101`. Hãy cho biết lệnh đầu tiên của chương trình thực sự nằm tại địa chỉ nào, và giải thích ý nghĩa của bit số 0 trong giá trị này. Điều gì sẽ xảy ra nếu bit đó bằng 0?

d. Liệt kê theo đúng thứ tự các công việc mà hàm `Reset_Handler` phải hoàn thành trước khi gọi `main()`. Với mỗi công việc, chỉ rõ nó phục vụ cho biến nào trong đoạn mã ở phần bối cảnh.

Câu 3. Thanh ghi PC và chu trình thực thi một lệnh (2 điểm)

a. Phát biểu chính xác nội dung mà thanh ghi PC (Program Counter — thanh ghi đếm chương trình, còn gọi là `R15`) lưu giữ tại một thời điểm bất kỳ, và mô tả ba pha của chu trình nạp lệnh — giải mã — thực thi (fetch–decode–execute).

b. Khi chương trình chạy tuần tự (không có lệnh rẽ nhánh), sau mỗi lệnh thì PC tăng thêm bao nhiêu byte? Vì sao trên Cortex-M câu trả lời không phải là một con số cố định duy nhất, khác với trường hợp vi điều khiển AVR 8-bit đã học?

c. Hoàn thành bảng sau, mô tả PC bị thay đổi như thế nào trong từng tình huống:

Tình huống	PC được cập nhật như thế nào?
Thực thi lệnh tuần tự	
Lệnh nhảy `B nhan`	
Lệnh gọi hàm `BL hamCon`	
Lệnh trở về `BX LR`	
Xảy ra một ngắt ngoại vi	

d. Trên Cortex-M3, nếu một lệnh đọc trực tiếp giá trị của PC thì kết quả nhận được lớn hơn địa chỉ của chính lệnh đang thực thi. Hãy giải thích nguyên nhân của hiện tượng này và cho biết độ lệch là bao nhiêu byte.

Câu 4. Ngăn xếp và các địa chỉ ô nhớ của nó (3 điểm)

a. Ngăn xếp trên Cortex-M được tổ chức theo nguyên tắc FILO/LIFO và thuộc loại "đầy, hướng xuống" (full descending). Hãy giải thích cụ thể: khi thực thi lệnh `PUSH`, thanh ghi `SP` được cộng hay trừ, thao tác ghi vào ô nhớ diễn ra trước hay sau khi thay đổi `SP`; và trả lời tương tự cho lệnh `POP`.

b. Cortex-M có hai con trỏ ngăn xếp là `MSP` và `PSP`. Nêu vai trò mặc định của từng con trỏ, cơ chế phần cứng nào quyết định con trỏ nào đang được dùng, và vì sao thiết kế này lại quan trọng khi hệ thống chạy hệ điều hành thời gian thực (RTOS).

c. Khi một hàm C được gọi, khung ngăn xếp (stack frame) của nó thường chứa những thành phần nào? Giải thích cụ thể vì sao địa chỉ trở về bắt buộc phải được cất vào ngăn xếp trong trường hợp hàm gọi lồng nhau, trong khi với một hàm lá (leaf function — không gọi hàm nào khác) thì có thể không cần.

d. Khi một ngắt xảy ra, phần cứng Cortex-M tự động đẩy 8 từ nhớ 32-bit vào ngăn xếp đang hoạt động. Hãy liệt kê 8 thanh ghi đó theo đúng thứ tự địa chỉ từ thấp đến cao, cho biết `SP` thay đổi bao nhiêu byte, và giải thích vì sao trong danh sách đó nhất thiết phải có PC.